AI-Powered Chess Engine

An Intelligent Agent using Adversarial Search

Minimax Algorithm | Alpha-Beta Pruning | Heuristic Evaluation | C# / WinForms

Adversarial Search · Artificial Intelligence · Game Theory

Project Overview

What we built and why

A fully functional Chess Engine built as an Intelligent Agent capable of playing against a human opponent — with no Machine Learning involved. Every decision is made through pure search and evaluation logic.

Intelligent Agent

The engine perceives the board state and selects the best possible move autonomously

High Performance

Alpha-Beta Pruning eliminates up to 90% of branches, enabling real-time play on standard hardware

Sound Evaluation

Custom heuristic scores material, position advantage, and king safety simultaneously

Framework & Technology Stack

Languages, tools, and architecture used

Language:	C# (.NET 10)
UI Layer:	Windows Forms (WinForms)
Paradigm:	Object-Oriented (OOP)
Threading:	async / Task.Run (no UI freeze)
Dev Tools:	VS Code + C# Dev Kit

Project Structure

```
📂 Models/
    Piece · Square · GameState
📂 Logic/
    MoveGenerator · MoveValidator
📂 AI/
    Evaluator · MinimaxEngine
📂 UI/
    BoardPanel · ChessForm
    Program.cs
```

Algorithm 1 — Minimax

Adversarial search for optimal decision-making

Minimax assumes both players play perfectly. The engine (Black) minimises the score while the human (White) maximises it. The AI searches all possible game continuations up to depth 3 and picks the move leading to the best guaranteed outcome.

Search Tree (Depth 3)

Pseudocode

```
minimax(board, depth,
        alpha, beta, isMax):

  if depth == 0:
    return evaluate(board)

  for each move:
    make move
    score = minimax(...)
    undo move

    update alpha/beta
    if β ≤ α → PRUNE ✂
```

Algorithm 2 — Alpha-Beta Pruning

Eliminating non-promising branches to speed up search

α

Alpha (α)

Best score the Maximiser (White) is guaranteed so far.
Starts at −∞, increases as better moves are found.

β

Beta (β)

Best score the Minimiser (Black) is guaranteed so far.
Starts at +∞, decreases as better moves are found.

```
Cutoff Rule:  if  β ≤ α  →  PRUNE the branch  (the opponent will never allow this path)
```

~90%

Branches eliminated
at best case

10×

Faster than plain
Minimax

3

Search depth
(medium difficulty)

MVV-LVA

Move ordering to
maximize pruning

Algorithm 3 — Heuristic Evaluation

Scoring a board position without searching to the end

Score = Material Value + Positional Bonus + King Safety

1. Material Value

Pawn	100
Knight	320
Bishop	330
Rook	500
Queen	900
King	20,000

centipawns (1 pawn = 100)

2. Positional Bonus

- Piece-Square Tables (8×8 grids)
- Knights rewarded in centre
- Bishops rewarded on long diagonals
- Rooks rewarded on open files
- King: hide in middlegame, centralise in endgame
- Endgame detection switches tables

3. King Safety

- Checks pawn shield (3 squares in front of king)
- Each missing shield pawn = −10
- Encourages castling early
- Prevents exposing king to attacks
- Critical in middlegame phase

How It Works — Game Flow

Step-by-step interaction between components

1. Human Move

Player clicks a piece then a destination.
BoardPanel highlights legal moves.

2. Move Validation

MoveValidator checks legality.
Filters moves leaving king in check.

3. State Update

GameState.ApplyMove() updates the board.
Handles castling, en passant, promotion.

4. AI Calculates

MinimaxEngine.GetBestMove() runs on a background thread (UI stays responsive).

5. Search + Prune

Minimax explores depth-3 tree.
Alpha-Beta cuts non-promising branches.

6. Best Move Applied

AI move is returned and applied.
Board redraws, turn flips back to human.

Features & User Interface

What the player sees and can do

🎨 **Visual Board**	8×8 grid with legal move highlights, check indicator, last move highlight
👆 **Click-to-Move**	Select a piece, green dots show legal destinations, click to move
⚠️ **Check Detection**	King square glows red when in check — both for player and AI
↩ **Undo Move**	Reverts the last two half-moves (AI + human) by replaying history
📊 **AI Stats Panel**	Live display of nodes evaluated and branches pruned per AI move
🔄 **Flip Board**	View the board from Black's perspective at any time
🏁 **Game Over Modal**	Detects checkmate, stalemate, and 50-move draw rule automatically

Board Preview

Nodes: 12,400 | Pruned: 8,900

Project Summary

Category	Details
Problem	Build an AI that plays chess at a human-competitive level without ML
Approach	Adversarial Search (Minimax) + Alpha-Beta Pruning
Evaluation	Material value + Piece-square tables + King safety heuristic
Language	C# (.NET 10) — strong typing, OOP, async threading
UI	Windows Forms — click-to-move, live AI stats, undo, flip board
Difficulty	Depth 3 search (Medium) — configurable to depth 2 (Easy) or 5 (Hard)
No ML Used	Pure algorithmic logic — no neural networks, no training data

A complete, playable Chess AI — built entirely on classical algorithms, OOP design, and clean C# code.